

Creating a Prototype for a Data Anonymization Service

MASTER THESIS

Tobias Kaiser

Submitted on 30 July 2026



Friedrich-Alexander-Universität Erlangen-Nürnberg
Faculty of Engineering, Department Computer Science
Professorship for Open Source Software

Supervisor:
Thomas Wolter
Prof. Dr. Dirk Riehle, M.B.A.



Friedrich-Alexander-Universität
Faculty of Engineering

Declaration of Originality

I confirm that the submitted thesis is original work and was written by me without further assistance. Appropriate credit has been given where reference has been made to the work of others. The thesis was not examined before, nor has it been published. The submitted electronic version of the thesis matches the printed version.

Erlangen, 30 July 2026

License

This work is licensed under the Creative Commons Attribution 4.0 International license (CC BY 4.0), see <https://creativecommons.org/licenses/by/4.0/>

Erlangen, 30 July 2026

Abstract

General instructions: If the thesis is written in German, this section contains the English translation. The abstract is 100–200 words maximum.

Contents

1	Introduction	1
2	Literature Review	3
2.1	Data Protection Laws	3
2.2	Server Design	3
2.3	Identity Management	3
3	Requirements	5
3.1	Functional Requirements	5
3.2	Nonfunctional Requirements	5
4	Architecture	9
4.1	Main System	9
4.1.1	Components	9
4.1.2	Dataflow	11
4.2	Anonymization System	12
4.3	Identity Management System	13
4.3.1	Frontend Dashboard	13
4.3.2	API	13
5	Design and Implementation	15
5.1	Compliance with Data Protection Laws	15
5.1.1	Reason for a Anonymization Service	15
5.1.2	Definition Anonymization and Pseudonymization	15
5.1.3	Approaches for Pseudonymization	15
5.1.4	Approaches for Anonymization	15
5.1.5	Deciding on an Approach	15
5.2	Anonymization Server	15
5.2.1	Implementing the Server	15
5.2.2	Implementing the Pseudonymization	15
5.2.3	Implementing the Anonymization	15
5.2.4	Deploying the Server	16

6	Evaluation	17
7	Conclusions	19
	References	21

List of Figures

4.1	Basic System Architecture	9
4.2	Main System Components ('MECOIS Github Repository', 2026) .	10
4.3	Dataflow ('MECOIS Github Repository', 2026)	11
4.4	Sortinhhat frontend (individuals are censored for privacy reasons)	13
4.5	Sortinhhat individuals section (uncensored) ('SortingHat's new interface', 2026)	14

List of Tables

3.1	Functional requirements	6
3.2	Nonfunctional requirements	7

Acronyms

DSGVO Datenschutz-Grundverordnung

GDPR General Data Protection Regulation

FLR functional legal requirements

FSR functional system requirements

FIR functional integration requirements

NFLR nonfunctional legal requirements

NFSR nonfunctional system requirements

NFIR nonfunctional integration requirements

LLM large language model

UUID universally unique identifier

1 Introduction

In the era of "Big Data," the ability to derive actionable insights from massive datasets is a primary driver of innovation across industries, from healthcare analytics to financial modeling. However, the unchecked accumulation of data presents a profound conflict: organizations are legally and ethically obligated to protect the privacy of individuals—under frameworks such as the General Data Protection Regulation (GDPR) and evolving global standards—while simultaneously needing to utilize that same data to train machine learning models and perform complex statistical analyses.

The market for data collection and processing is huge. Companies, law enforcement and even your employer can find great use in information about their customer, potential criminal or employee. Advertisements are more effective, if they are only served to future customers. Dynamic Pricing tries to set your personal price as high as possible to maximize profits. Your bill might be influenced by your income, spending behavior, interest in the product category or nationality. This may even continue at your job. Bosses want the most productive workers, which is a difficult metric to measure. This incentivizes enterprises to provide tools, that can quantify how valuable an employee is.

1. Introduction

2 Literature Review

2.1 Data Protection Laws

2.2 Server Design

2.3 Identity Management

2. Literature Review

3 Requirements

The purpose of this chapter is to define the requirements necessary to facilitate a GDPR-compliant productivity benchmarking system. ‘ISO/IEC/IEEE International Standard - Systems and software engineering–Vocabulary’ (2017) splits requirements into functional and nonfunctional ones. Functional requirements define what a product must do and nonfunctional ones declare how the system should operate. Furthermore, the requirements can be classified into legality, system and integration. The service must abide to relevant data protection laws. System requirements specify how the service itself will be designed according to the author. The MECOIS project itself sets some integration demands in its contributing guidelines (‘MECOIS Github Repository’, 2026), that must be satisfied.

3.1 Functional Requirements

The table 3.1 specifies what the anonymization must do. They are sorted by functional legal requirements (FLR), functional system requirements (FSR) and functional integration requirements (FIR).

The Data Anonymization requirement FLR-1 is defined very vague by design. There can be many ways to anonymize data according to the DSGVO. Chapter 5 will compare and present different approaches. It stands in direct conflict with FSR-1 Preserve Data. The aim is to find a compromise between these two.

3.2 Nonfunctional Requirements

The table 3.2 specifies how the anonymization service must operate. They are sorted by nonfunctional system requirements (NFSR) and nonfunctional integration requirements (NFIR). There are no nonfunctional legal requirements (NFLR), because it is evaluated in chapter 5, how the legal requirements will be satisfied.

Table 3.1: Functional requirements

ID	Requirement	Description
FLR-1	Data Anonymization	The service must follow the GDPR which is defined by the Datenschutz-Grundverordnung (DSGVO) in Germany.
FSR-1	Preserve Data	The service must preserve all available Data.
FSR-2	independent Deployment	The service can be deployed and run independent from the rest of the MECOIS project.
FIR-1	Seamless Integration	The service must integrate into the project with as few changes as possible to existing features.

Table 3.2: Nonfunctional requirements

ID	Requirement	Description
NFSR-1	Containerized Service	The service must run in a docker container.
NFIR-1	Licence Policy	The contributing guidelines (‘MECOIS Github Repository’, 2026) state: ”You are allowed to use any package that can help you do your work, as long as it is not a copyleft or proprietary license. However, it is still a good idea to ask beforehand, so you do not put in unnecessary work.”
NFIR-2	Styleguide and Clean Code	The contributing guidelines (‘MECOIS Github Repository’, 2026) state: ”We follow the PEP8 standard (with minor exceptions like allowing longer line limits). You can run the custom linter using: pylint. Additionally, your code must follow these rules before being merged into the main branch: • Remove comments that are not necessary for understanding your code • Remove unnecessary print statements • Make sure to update and provide the DocString description of the methods you write. Make sure to have it adjusted to your use-case if you copy method-stubs from other part of the code”

3. Requirements

4 Architecture

The MECOIS setup consists of 3 separate systems, that communicate via HTTP (see Figure 4.1). The systems are separated to protect sensitive information.

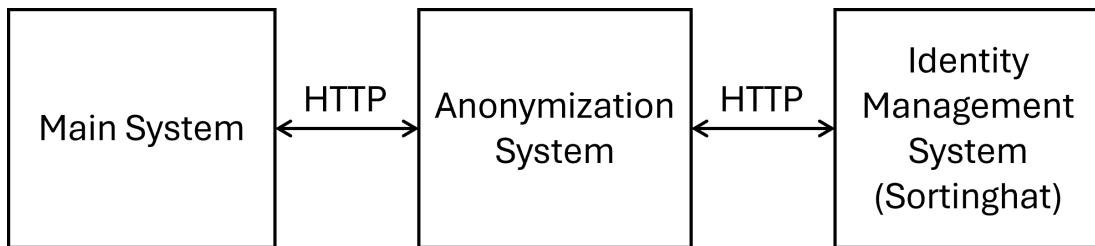


Figure 4.1: Basic System Architecture

4.1 Main System

The main systems is responsible everything but the identity management and therefore has many components.

4.1.1 Components

Figure 4.2 displays an overview of all components. (‘MECOIS Github Repository’, 2026)

Sources

The sources define the origin for the data. Currently we are using Git (Github and Gitlab), Confluence and Jira as source. For Example from Github we pull among other commits, issues and pipelines.

Infrastructure

The infrastructure provides the database for the project. The data is stored in a datalake, which is a repository, where data is stored as-is without further

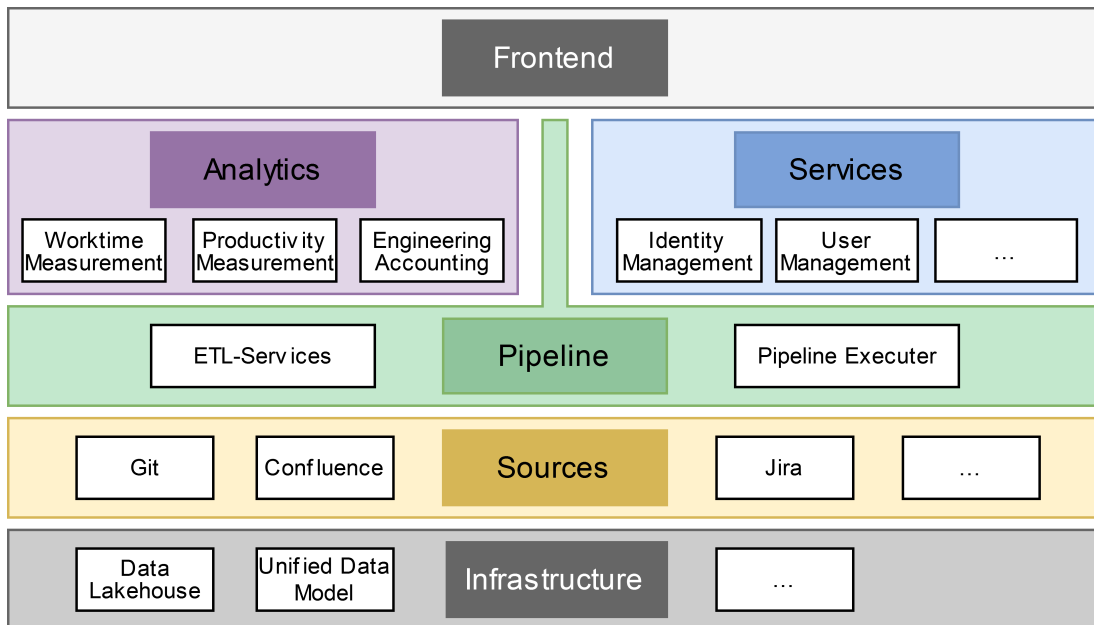


Figure 4.2: Main System Components (‘MECOIS Github Repository’, 2026)

structure (‘What is a data lake?’, 2026). Each set of data passes through 4 Stages: raw, bronze, silver, gold.

Pipeline

The pipelines transform the data between the stages.

Analytics

In the last step the data is used to provide multiple analytics for customers. Examples for such analytics are:

- Calculating the Costs of Inner Source Collaboration by Computing the Time Worked (Buchner & Riehle, 2021)
- Estimating Software Contribution Time from Git and GitHub Metadata (Stenzel, 2026)

Services

Services are helper modules for pipelines and analytics. Such helper modules are:

- AI: a framework for communicating with a large language model (LLM).
- config handling: Parses configuration files for pipelines.

Frontend

The frontend provides a Dashboard for the user-facing components of MECOIS.

Orchestrators

The orchestrators can be used to run the pipeline. The set up the configuration and run the pipeline for the selected sources.

4.1.2 Dataflow

After pulling the source the raw data is stored exactly as provided as JSON. In the datalake the Data is processed multiple times and the states are called bronze, silver and gold. The processing is done by the pipelines.

Each pipeline has 3 steps:

1. Read the data from the datalake
2. transform data
3. write the transformed data to the datalake

Figure 4.3 displays an overview of all components. (‘MECOIS Github Repository’, 2026)

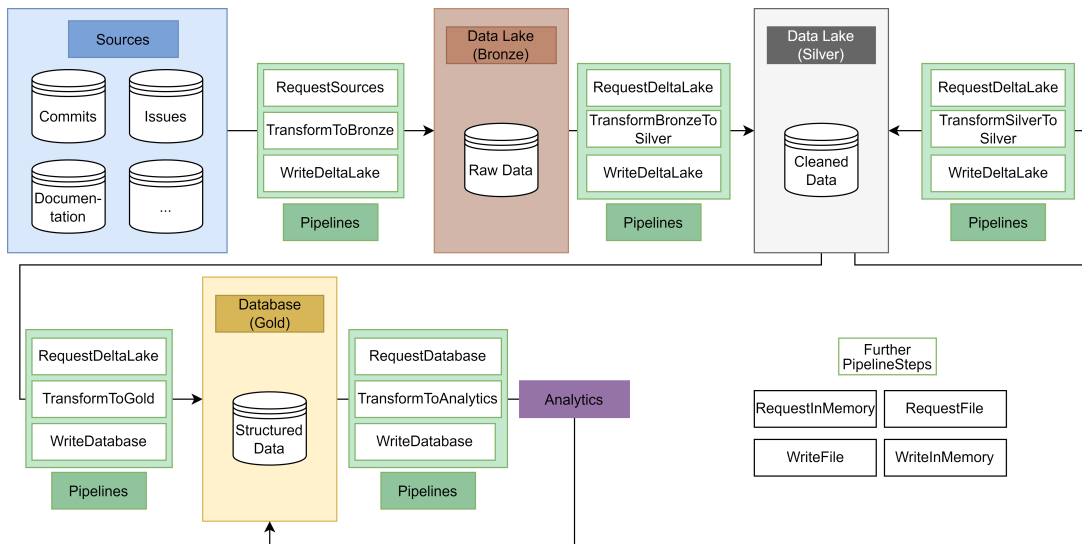


Figure 4.3: Dataflow (‘MECOIS Github Repository’, 2026)

JSON to Bronze

The `JSONtoBronzePipeline` processes the raw JSON data provided by the sources and writes it unchanged to the Datalake. The data itself isn’t changed in any

way. Each source has an own table.

Bronze to Silver

The `BronzeToSilverPipeline` processes the bronze data in multiple steps to be ready for the analytics.

Flatten table The bronze state cannot represent nested data. (e.g. a Github commit contains an autor and the author itself has multiple properties). In the first step those nested data in columns is split in multiple columns, so that each on only contains a single datapoint.

Drop irrelevant columns Not all data is useful. Some columns (e.g. the URL to the avatar of a Github user) are dropped.

Fix dates Dates are still stored as String. This steps casts these Strings into

Anonymize Data The data is sent to the anonymization service for processing.

add UUID Each table has a way to create an universally unique identifier (UUID) by concatenating some columns and hashing the result.

Silver to Silver

Identities may update in the identity mangement system. The `UpdateIdentityPipeline` updates any identities in the datalake that have been updated in the Sortinghat.

Silver to Gold

There is no single `Silver2GoldPipeline`. The analytics represent the gold to silver pipelines. Each analytic uses the silver data to provide some metric.

Gold to Gold

Some analytics may need some mechanism to update the generated metric. This process is defined as gold to gold transformation.

4.2 Anonymization System

The anonymization system provides the anonymization service, that anonymizes the data according to the GDPR. It provides a client library for sending data and requesting identities.

4.3 Identity Management System

The identity management system is responsible for storing all information about individuals. The project is using the tool Sortinghat from the toolset GrimoireLab by CHAOSS. This application is provided under the GPL-3.0 license.

One individual may have multiple identities on different platforms. The system can pool these different accounts into one individual. These individuals can also be grouped into teams. Multiple teams can form an organization. (‘SortingHat’, 2026) (‘SortingHat Github Repository’, 2026)

4.3.1 Frontend Dashboard

The Sortinghat provides a dashboard, where individuals can be merged and associated into teams and organizations.

Figure 4.4 shows a screenshot of the frontend. Since the screenshot contains real data, the individuals are blacked out. The individuals section can be seen in figure 4.5.

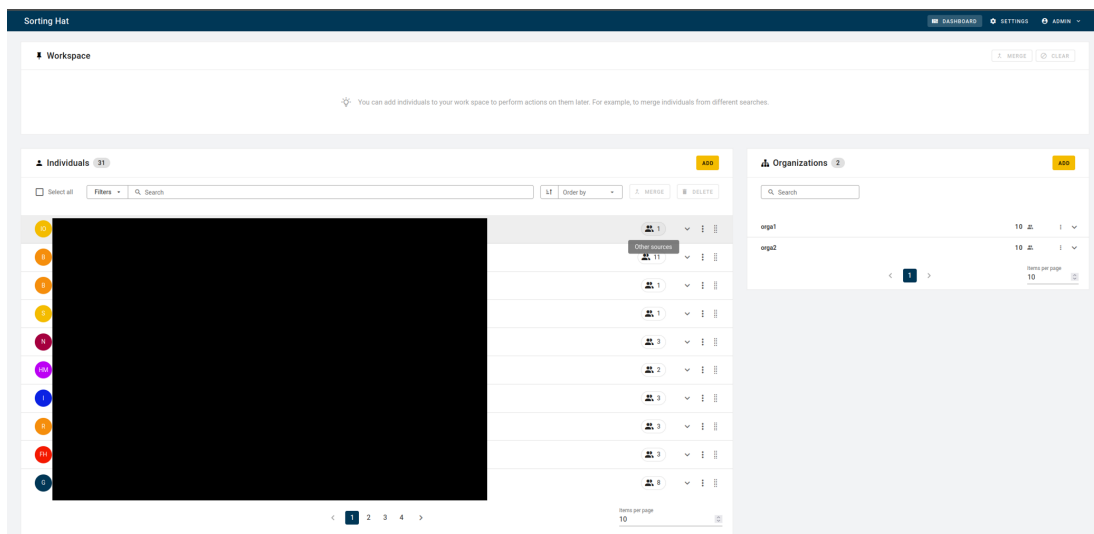


Figure 4.4: Sortinhhat frontend (individuals are censored for privacy reasons)

4.3.2 API

The Sortinghat also provided an API for automating interactions, like listing, updating and deleting individuals.

4. Architecture

Individuals

9

ADD

☐

Select all

Filters

Search

Name

MERGE

DELETE

EM

Eva Millán

evamillanmartin@gmail.com

Georg J.P. Link

linkgeorg@gmail.com

JS

John Smith

johnsmith@gmail.com

Quan Zhou

quan@bitergia.com

Santiago Dueñas

sduenas@bitergia.com

VS

Veerasamy Sevagen

amFOSS

sevagenv@gmail.com

VS

Veerasamy Sevagen

sevagenv@gmail.com

Venu Vardhan Reddy Tekula

venuvardhanreddytekula8@gmail.com

VT

Venu Vardhan Reddy Tekula

venu@bitergia.com

Figure 4.5: Sortinhat individuals section (uncensored) (‘SortingHat’s new interface’, 2026)

5 Design and Implementation

Chapter 4 described the architecture of the MECOIS project and the integration of the anonymization service. This chapter will explain how the service is implemented.

5.1 Compliance with Data Protection Laws

Before implementing the service, the approach to the anonymization has to be discussed.

5.1.1 Reason for a Anonymization Service

problem: Zweckbindung der gespeicherten Daten.

5.1.2 Definition Anonymization and Pseudonymization

5.1.3 Approaches for Pseudonymization

5.1.4 Approaches for Anonymization

5.1.5 Deciding on an Approach

5.2 Anonymization Server

5.2.1 Implementing the Server

5.2.2 Implementing the Pseudonymization

TODO: überschreibt mit der eigentlichen Methode austauschen

5.2.3 Implementing the Anonymization

TODO: überschreibt mit der eigentlichen Methode austauschen

5.2.4 Deploying the Server

Docker Container

6 Evaluation

lorem ipsum

7 Conclusions

lorem ipsum

7. Conclusions

References

- Buchner, S., & Riehle, D. (2021). Calculating the costs of inner source collaboration by computing the time worked. https://dirkriehle.com/wp-content/uploads/2021/09/Buchner_Riehle_LOCAL-VERSION__Computing_the_Costs_of_Inner_Source_Collaboration.pdf
- Iso/iec/ieee international standard - systems and software engineering-vocabulary. (2017). *ISO/IEC/IEEE 24765:2017(E)*, 1–541. <https://doi.org/10.1109/IEEESTD.2017.8016712>
- Mecois github repository*. (2026). MECOIS. Retrieved July 8, 2026, from <https://github.com/Mecois/mecois>
- Sortinghat*. (2026). GrimoireLab. Retrieved July 14, 2026, from <https://chaoss.github.io/grimoirelab-tutorial/sortinghat>
- Sortinghat github repository*. (2026). GrimoireLab. Retrieved July 14, 2026, from <https://github.com/chaoss/grimoirelab-sortinghat>
- Sortinghat's new interface*. (2026). GrimoireLab. Retrieved July 14, 2026, from <https://chaoss.github.io/grimoirelab-tutorial/docs/sortinghat/interface/>
- Stenzel, M. (2026). Estimating software contribution time from git and github metadata. https://oss.cs.fau.de/wp-content/uploads/2026/04/Stenzel_2026.pdf
- What is a data lake?* (2026). AWS. Retrieved July 9, 2026, from <https://aws.amazon.com/what-is/data-lake/>